

```
In [2]: import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Input, Dropout
from tensorflow.keras.applications import VGG19
from tensorflow.keras.applications.vgg19 import preprocess_input
from tensorflow.keras.utils import to_categorical

N_TRAIN = 5000
N_TEST = 1000
IMG_SIZE = 224
BATCH_SIZE = 32
EPOCHS = 15
SEED = 42

tf.random.set_seed(SEED)
np.random.seed(SEED)

(x_train, y_train), (x_test, y_test) = mnist.load_data()

if N_TRAIN is not None:
    idx = np.random.permutation(len(x_train))[:N_TRAIN]
    x_train, y_train = x_train[idx], y_train[idx]

if N_TEST is not None:
    idx = np.random.permutation(len(x_test))[:N_TEST]
    x_test, y_test = x_test[idx], y_test[idx]

x_test_gray = x_test.copy()
y_test_labels = y_test.copy()

# Add channel dimension, keep as uint8 to save memory
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

print(f"Train: {x_train.shape} Test: {x_test.shape}")

def preprocess_image(image, label):
    image = tf.cast(image, tf.float32)
    image = tf.image.grayscale_to_rgb(image)
    image = tf.image.resize(image, (IMG_SIZE, IMG_SIZE))
    image = preprocess_input(image)
    return image, label
```

```

def make_dataset(x, y):
    ds = tf.data.Dataset.from_tensor_slices((x, y))
    ds = ds.map(preprocess_image, num_parallel_calls=tf.data.AUTOTUNE)
    return ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

train_dataset = make_dataset(x_train, y_train)
test_dataset = make_dataset(x_test, y_test)

base_model = VGG19(
    weights="imagenet",
    include_top=False,
    input_shape=(IMG_SIZE, IMG_SIZE, 3)
)
base_model.trainable = False

feature_extractor = Sequential([
    base_model,
    GlobalAveragePooling2D()
])

print("\nExtracting training features...")
train_features = feature_extractor.predict(train_dataset, verbose=1)

print("Extracting test features...")
test_features = feature_extractor.predict(test_dataset, verbose=1)

print(f"Feature shape: {train_features.shape}") # (N, 512)

FEATURE_DIM = train_features.shape[1]

def build_head(units):
    return Sequential([
        Input(shape=(FEATURE_DIM,)),
        Dense(units, activation="relu"),
        Dropout(0.3),
        Dense(10, activation="softmax")
    ])

def train_head(units, name):
    print(f"\n{'=' * 40}\nTraining {name} (Dense {units})\n{'=' * 40}")
    model = build_head(units)
    model.compile(
        optimizer="adam",
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )
    history = model.fit(
        train_features, y_train,
        epochs=EPOCHS,
        batch_size=BATCH_SIZE,
        validation_split=0.1,
        shuffle=True,

```

```

        verbose=1
    )
    loss, acc = model.evaluate(test_features, y_test, verbose=0)
    return model, history, loss, acc

model1, hist1, loss1, acc1 = train_head(128, "MODEL 1")
model2, hist2, loss2, acc2 = train_head(256, "MODEL 2")

pred1 = model1.predict(test_features[:2], verbose=0)
pred2 = model2.predict(test_features[:2], verbose=0)

def show_results(name, preds, accuracy, loss):
    print(f"\n{'=' * 30}\n{name} RESULTS\n{'=' * 30}")
    print("Accuracy:", round(accuracy * 100, 2), "%")
    print("Loss:", round(loss, 4))

    plt.figure(figsize=(8, 4))

    for i in range(2):
        predicted = preds[i].argmax()
        actual = y_test_labels[i]
        vector = y_test[i].astype(int)
        vector_text = "[" + ", ".join(map(str, vector)) + "]"

        plt.subplot(1, 2, i + 1)

        plt.imshow(x_test_gray[i], cmap="gray")
        plt.title(
            f"Predicted: {predicted}\n"
            f"Actual: {actual}\n"
            f"One-Hot: {vector_text}",
            fontsize=9
        )
        plt.axis("off")

    plt.suptitle(f"{name} Predictions")
    plt.tight_layout()
    plt.show()

show_results("MODEL 1", pred1, acc1, loss1)
show_results("MODEL 2", pred2, acc2, loss2)

print(f"\n{'=' * 40}")
print("COMPARISON")
print(f"\n{'=' * 40}")
print(f"Model 1 (Dense 128): {acc1 * 100:.2f}% loss {loss1:.4f}")
print(f"Model 2 (Dense 256): {acc2 * 100:.2f}% loss {loss2:.4f}")

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(hist1.history["val_accuracy"], label="Model 1 (128)")
plt.plot(hist2.history["val_accuracy"], label="Model 2 (256)")

```

```
plt.title("Validation Accuracy")
plt.xlabel("Epoch")
plt.legend()
plt.grid(alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(hist1.history["val_loss"], label="Model 1 (128)")
plt.plot(hist2.history["val_loss"], label="Model 2 (256)")
plt.title("Validation Loss")
plt.xlabel("Epoch")
plt.legend()
plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()
```

Train: (5000, 28, 28, 1) Test: (1000, 28, 28, 1)

Extracting training features...

157/157 [=====] - 338s 2s/step

Extracting test features...

32/32 [=====] - 68s 2s/step

Feature shape: (5000, 512)

=====

Training MODEL 1 (Dense 128)

=====

WARNING:tensorflow:From C:\Users\Student\AppData\Roaming\Python\Python310\site-packages\keras\src\optimizers_init_.py:309: The name tf.train.Optimizer is deprecated. Please use tf.compat.v1.train.Optimizer instead.

Epoch 1/15

WARNING:tensorflow:From C:\Users\Student\AppData\Roaming\Python\Python310\site-packages\keras\src\utils\tf_utils.py:492: The name tf.ragged.RaggedTensorValue is deprecated. Please use tf.compat.v1.ragged.RaggedTensorValue instead.

WARNING:tensorflow:From C:\Users\Student\AppData\Roaming\Python\Python310\site-packages\keras\src\engine\base_layer_utils.py:384: The name tf.executing_eagerly_outside_functions is deprecated. Please use tf.compat.v1.executing_eagerly_outside_functions instead.

141/141 [=====] - 1s 2ms/step - loss: 0.9413 - accuracy: 0.722 - val_loss: 0.2680 - val_accuracy: 0.9200

Epoch 2/15

141/141 [=====] - 0s 1ms/step - loss: 0.2889 - accuracy: 0.9096 - val_loss: 0.1931 - val_accuracy: 0.9400

Epoch 3/15

141/141 [=====] - 0s 1ms/step - loss: 0.2011 - accuracy: 0.9391 - val_loss: 0.1477 - val_accuracy: 0.9540

Epoch 4/15

141/141 [=====] - 0s 1ms/step - loss: 0.1460 - accuracy: 0.9549 - val_loss: 0.1432 - val_accuracy: 0.9500

Epoch 5/15

141/141 [=====] - 0s 1ms/step - loss: 0.1216 - accuracy: 0.9622 - val_loss: 0.1212 - val_accuracy: 0.9680

Epoch 6/15

141/141 [=====] - 0s 1ms/step - loss: 0.0969 - accuracy: 0.9709 - val_loss: 0.1303 - val_accuracy: 0.9660

Epoch 7/15

141/141 [=====] - 0s 1ms/step - loss: 0.0772 - accuracy: 0.9769 - val_loss: 0.1102 - val_accuracy: 0.9660

Epoch 8/15

141/141 [=====] - 0s 1ms/step - loss: 0.0691 - accuracy: 0.9782 - val_loss: 0.1141 - val_accuracy: 0.9640

Epoch 9/15

141/141 [=====] - 0s 1ms/step - loss: 0.0483 - accuracy: 0.9880 - val_loss: 0.1172 - val_accuracy: 0.9620

Epoch 10/15

141/141 [=====] - 0s 1ms/step - loss: 0.0521 - accuracy: 0.9836 - val_loss: 0.1088 - val_accuracy: 0.9680

Epoch 11/15

141/141 [=====] - 0s 1ms/step - loss: 0.0412 - accuracy: 0.9880 - val_loss: 0.0926 - val_accuracy: 0.9680

Epoch 12/15

141/141 [=====] - 0s 1ms/step - loss: 0.0343 - accuracy: 0.9909 - val_loss: 0.1078 - val_accuracy: 0.9620

```
Epoch 13/15
141/141 [=====] - 0s 2ms/step - loss: 0.0301 - accuracy: 0.9
913 - val_loss: 0.0990 - val_accuracy: 0.9720
Epoch 14/15
141/141 [=====] - 0s 1ms/step - loss: 0.0328 - accuracy: 0.9
916 - val_loss: 0.1083 - val_accuracy: 0.9660
Epoch 15/15
141/141 [=====] - 0s 1ms/step - loss: 0.0353 - accuracy: 0.9
893 - val_loss: 0.1095 - val_accuracy: 0.9700

=====
Training MODEL 2 (Dense 256)
=====
Epoch 1/15
141/141 [=====] - 1s 2ms/step - loss: 0.8173 - accuracy: 0.7
704 - val_loss: 0.1832 - val_accuracy: 0.9420
Epoch 2/15
141/141 [=====] - 0s 2ms/step - loss: 0.2151 - accuracy: 0.9
342 - val_loss: 0.1569 - val_accuracy: 0.9560
Epoch 3/15
141/141 [=====] - 0s 2ms/step - loss: 0.1374 - accuracy: 0.9
569 - val_loss: 0.1271 - val_accuracy: 0.9600
Epoch 4/15
141/141 [=====] - 0s 2ms/step - loss: 0.1054 - accuracy: 0.9
669 - val_loss: 0.0969 - val_accuracy: 0.9680
Epoch 5/15
141/141 [=====] - 0s 2ms/step - loss: 0.0829 - accuracy: 0.9
729 - val_loss: 0.1138 - val_accuracy: 0.9680
Epoch 6/15
141/141 [=====] - 0s 2ms/step - loss: 0.0600 - accuracy: 0.9
820 - val_loss: 0.0995 - val_accuracy: 0.9700
Epoch 7/15
141/141 [=====] - 0s 2ms/step - loss: 0.0496 - accuracy: 0.9
833 - val_loss: 0.1059 - val_accuracy: 0.9660
Epoch 8/15
141/141 [=====] - 0s 2ms/step - loss: 0.0427 - accuracy: 0.9
858 - val_loss: 0.1025 - val_accuracy: 0.9740
Epoch 9/15
141/141 [=====] - 0s 2ms/step - loss: 0.0276 - accuracy: 0.9
918 - val_loss: 0.1091 - val_accuracy: 0.9680
Epoch 10/15
141/141 [=====] - 0s 2ms/step - loss: 0.0280 - accuracy: 0.9
916 - val_loss: 0.1198 - val_accuracy: 0.9640
Epoch 11/15
141/141 [=====] - 0s 2ms/step - loss: 0.0301 - accuracy: 0.9
918 - val_loss: 0.1122 - val_accuracy: 0.9720
Epoch 12/15
141/141 [=====] - 0s 2ms/step - loss: 0.0134 - accuracy: 0.9
976 - val_loss: 0.1041 - val_accuracy: 0.9680
Epoch 13/15
141/141 [=====] - 0s 2ms/step - loss: 0.0229 - accuracy: 0.9
922 - val_loss: 0.1124 - val_accuracy: 0.9640
Epoch 14/15
141/141 [=====] - 0s 2ms/step - loss: 0.0174 - accuracy: 0.9
967 - val_loss: 0.1078 - val_accuracy: 0.9660
Epoch 15/15
141/141 [=====] - 0s 2ms/step - loss: 0.0174 - accuracy: 0.9
942 - val_loss: 0.1085 - val_accuracy: 0.9700

=====
```

MODEL 1 RESULTS

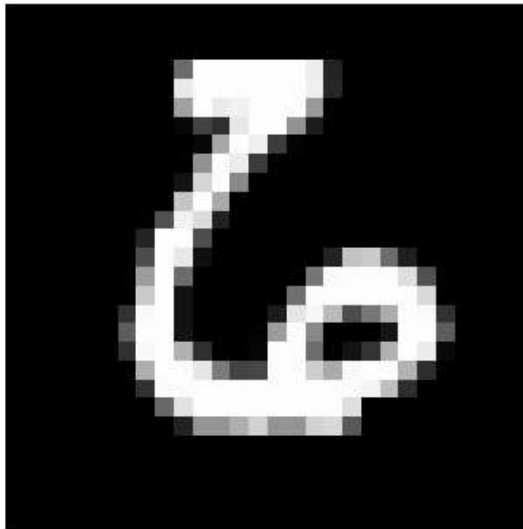
=====

Accuracy: 96.1 %

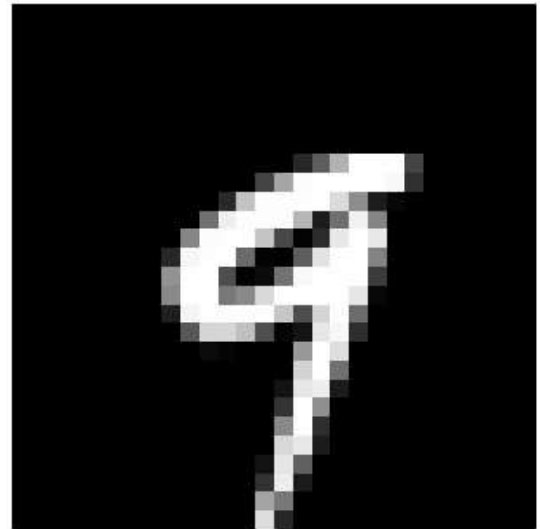
Loss: 0.1117

MODEL 1 Predictions

Predicted: 6
Actual: 6
One-Hot: [0,0,0,0,0,0,1,0,0,0]



Predicted: 9
Actual: 9
One-Hot: [0,0,0,0,0,0,0,0,0,1]



=====

MODEL 2 RESULTS

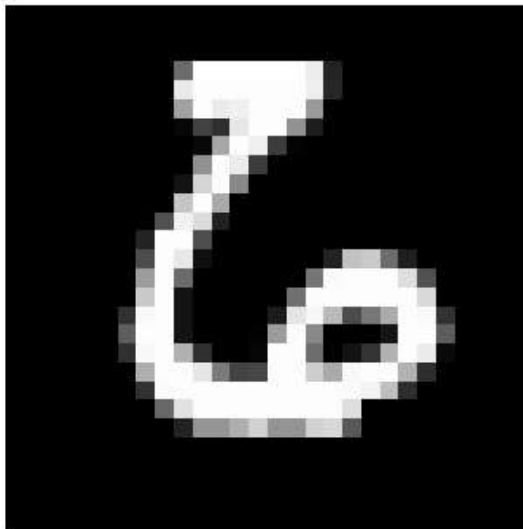
=====

Accuracy: 96.3 %

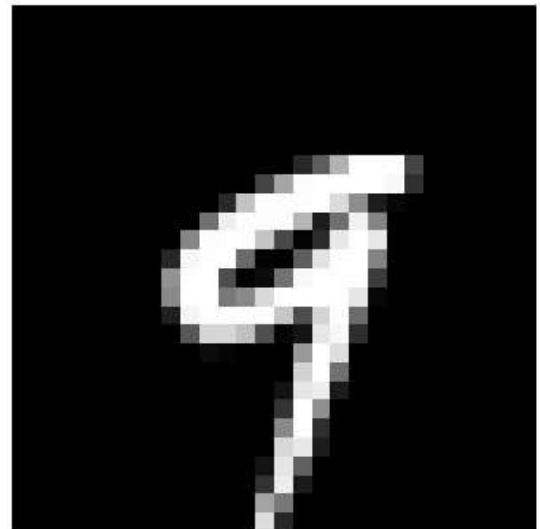
Loss: 0.1179

MODEL 2 Predictions

Predicted: 6
Actual: 6
One-Hot: [0,0,0,0,0,0,1,0,0,0]



Predicted: 9
Actual: 9
One-Hot: [0,0,0,0,0,0,0,0,0,1]



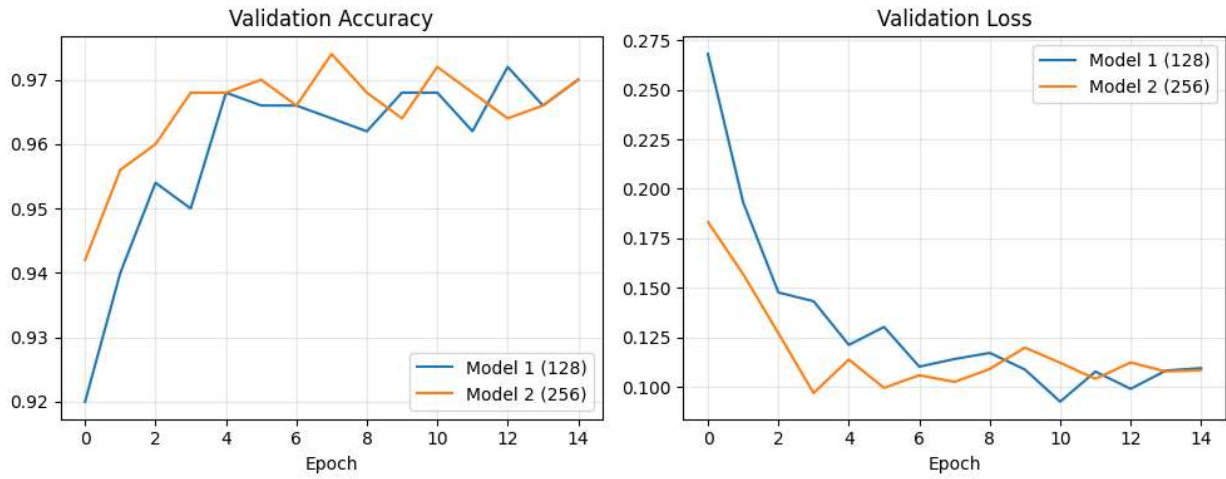
=====

COMPARISON

=====

Model 1 (Dense 128): 96.10% loss 0.1117

Model 2 (Dense 256): 96.30% loss 0.1179



```
In [ ]:
```